

Modular Monolith Architecture

1) Build intuition

Start with the core idea

A **monolith** means:

- one codebase (or one deployable application)
- one runtime/process group for the app
- usually one database
- one deployment unit

A **modular monolith** means:

- still **one deployable application**
- but inside that application, the code is split into **strict business modules**
- each module has **clear boundaries, its own models/use-cases**, and ideally **its own owned data access**

So the intuition is:

A modular monolith is a monolith that is designed like a set of internal mini-systems, not a pile of files.

The mental model

Think of the system like an apartment building:

- The **building** is the monolith.
- Each **apartment** is a module like User, Order, Billing, Catalog.
- Apartments live in the same building, share some common infrastructure (electricity/water = logging/auth/config), but **people should enter through proper doors**, not by breaking walls between apartments.

If you don't enforce those walls, a monolith becomes a **big ball of mud**:

- everyone imports everything
- modules query each other's tables directly
- business logic leaks everywhere
- refactoring becomes scary

A modular monolith prevents that by saying:

- **you can live in one deployable**

- but **you must behave as if boundaries matter**

That is the key intuition.

Why senior engineers like it

Because it gives you much of what teams actually need:

- simple deployment
- simple local development
- simple debugging
- low operational complexity
- strong transactional consistency
- and still good code organization if designed properly

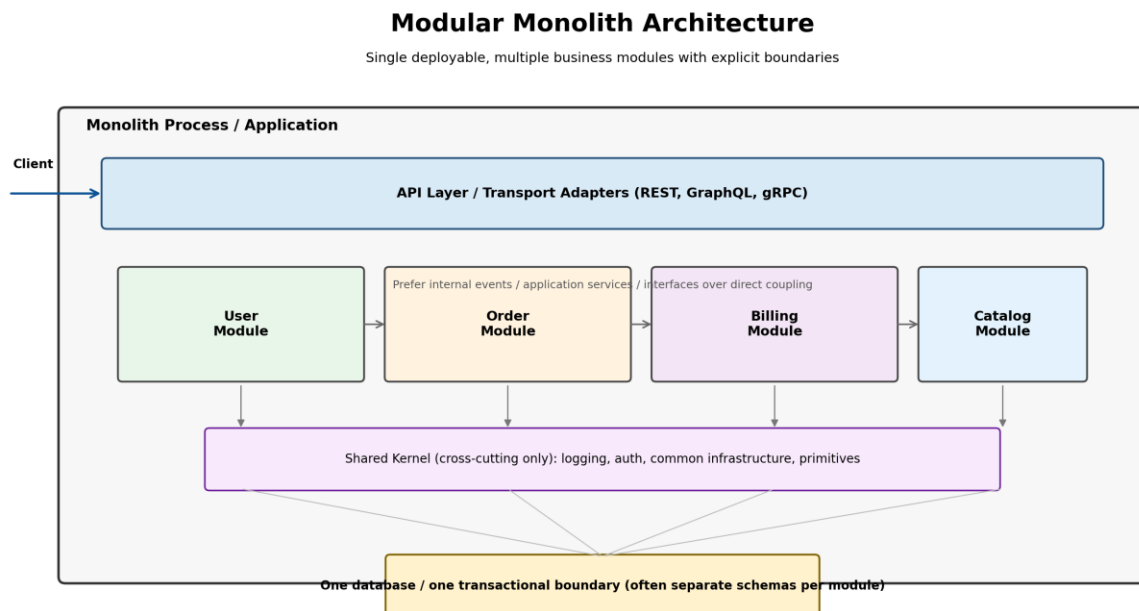
In other words:

It optimizes for software design first, and distributed systems later.

That is usually the right trade-off early and mid-stage.

2) Describe the architecture

Here is a visual model of a modular monolith:



What this diagram means

A. One application process

All modules run inside the same application boundary.

Examples:

- one NestJS app
- one Spring Boot app
- one ASP.NET Core app

Even if the app internally has many modules, from deployment perspective it is still **one system**.

B. Modules represent business capabilities

Typical modules might be:

- Identity
- User
- Catalog
- Order
- Payment
- Billing
- Notification

Each module should ideally contain:

- domain entities / aggregates
- use cases / application services
- repositories/interfaces
- controllers or handlers
- validation rules
- persistence implementation
- internal events

Each module should answer:

“What business capability does this module own?”

If the answer is fuzzy, the boundaries are weak.

C. Communication between modules is controlled

Module-to-module communication should happen through:

- explicit interfaces
- application services
- domain events/internal events
- command/query handlers

Not through:

- random imports
- shared mutable state
- direct table access into another module's persistence
- copying logic from another module

The rule is:

A module owns its behavior and data access. Other modules ask through a contract.

D. Shared kernel exists, but it must stay small

Most modular monoliths have some shared components:

- logging
- auth primitives
- config
- base exceptions
- monitoring
- event bus abstraction
- common value objects
- infrastructure wrappers

But this shared area is dangerous.

If it grows too much, it becomes a dumping ground and all modules become coupled through “shared”.

So:

Shared kernel should contain cross-cutting concerns, not business logic.

Bad examples to place in shared:

- OrderStatusLogic
- UserDiscountPolicy

- PaymentEligibilityRules

Those belong in business modules, not common.

E. Usually one database, but module ownership still matters

A modular monolith often uses **one database** for simplicity and strong consistency.

But “one database” does **not** mean “everyone touches every table.”

Good practice:

- each module owns its tables
- if possible, use separate schemas per module
- only the owner module should write/read directly
- other modules access through APIs/services/events

So even with one DB, data governance should feel like:

- orders.* owned by Order module
- billing.* owned by Billing module
- users.* owned by Identity/User module

That discipline makes later extraction to microservices much easier.

3) Steps to design a modular monolith backend (with industry best practices)

I’ll give this in a practical order, as if you were designing a real backend from scratch.

Step 1: Start with business capabilities, not technical layers

Best practice

Split by **business domain / feature**, not by technical folders like:

- /controllers
- /services
- /repositories
- /entities

That “layer-first” structure often causes unrelated business rules to mix together.

Instead do:

```
1  src/  
2    modules/  
3      identity/  
4      users/  
5      catalog/  
6      orders/  
7      billing/  
8      notifications/
```

Each module should contain its own internals.

Why this is industry-standard thinking

This aligns with:

- **DDD bounded contexts**
- **package/module by feature**
- **high cohesion, low coupling**
- **ownership-based architecture**

Common anti-pattern

“Everything is separated by layer globally.”

That gives a technically neat project tree but poor business boundaries.

Step 2: Identify bounded contexts

Ask:

- What are the major business capabilities?
- Which concepts mean different things in different parts of the system?
- Which workflows belong together?

Example in e-commerce:

- Catalog manages products, categories, prices (display side)
- Order manages carts, order placement, order state transitions
- Billing manages invoices, payment records, settlements
- Identity handles auth/session/roles

The same word can mean different things across contexts.

For example, **customer status** in CRM may be different from **user role** in Identity.

Best practice

Design modules around **business consistency boundaries**, not database tables.

Common anti-pattern

“Let’s make one module per entity/table.”

That creates an anemic, CRUD-driven system, not a business architecture.

Step 3: Define ownership clearly

For each module, define:

- what it owns
- what data it owns
- what operations others may call
- what events it emits
- what it must never expose directly

Example:

Order module owns

- create order
- validate line items
- reserve stock (through a contract/event)
- manage order lifecycle

Order module should not expose

- its internal ORM entities to other modules
- direct repository access
- raw DB tables for others to query

Best practice

Create a short contract doc or ADR per module:

- responsibilities
- public interface
- dependencies
- data ownership
- invariants

This is simple, but extremely powerful in large teams.

Step 4: Define allowed dependencies between modules

This is one of the most important design steps.

You must decide:

- which modules can depend on which
- whether dependencies are directional
- whether communication is sync or async

A healthy dependency graph is usually:

- acyclic if possible
- stable in direction
- centered around business flow, not convenience

Example

Order -> Catalog may query product snapshot rules

Order -> Billing may request invoice creation

But Catalog should not suddenly depend on Billing

Best practice

Enforce dependency rules via:

- module public APIs only
- linting or import rules
- package visibility
- team conventions reviewed in PRs

Common anti-pattern

“Any module can import anything because it’s all one codebase.”

That destroys modularity.

Step 5: Design the inside of each module using clean boundaries

Inside a module, a strong structure is often:

```
1  orders/
2    api/           # controllers / handlers / DTOs
3    application/   # use cases, command handlers, query
handlers
4    domain/        # entities, aggregates, value objects,
domain rules
```


5	infrastructure/	# ORM, repositories, adapters
6	contracts/	# public module interfaces

This is not mandatory, but the pattern is good because it separates:

- transport concerns
- use-case orchestration
- domain rules
- infrastructure

Best practice

Use **Clean Architecture / Hexagonal ideas inside each module**, even if the whole app is one deployable.

Why?

Because it makes modules easier to test, extract, and reason about.

Step 6: Protect the database boundary

A modular monolith often fails not at code level, but at DB level.

The common failure mode:

- modules avoid each other's code
- but directly join each other's tables everywhere

Then the real architecture becomes the database, not the modules.

Best practice

Follow these rules:

1. A module owns its tables.
2. Other modules do not directly mutate those tables.
3. Cross-module reads should happen through:
 - public application APIs
 - dedicated query services
 - read models/projections
4. If possible, separate schemas per module.

Common anti-pattern

"Just do one join across 8 tables from 4 domains because it's convenient."

That creates invisible coupling that becomes painful later.

Step 7: Choose the right module communication style

In a modular monolith, module interaction can be:

Synchronous (in-process)

- service-to-service calls within the application
- simple, fast, easy to debug

Good for:

- validation
- immediate rules
- user-facing request flows

Asynchronous (internal events)

- OrderPlaced
- InvoiceCreated
- UserRegistered

Good for:

- side effects
- notifications
- reporting/projections
- eventual workflows

Best practice

Use:

- **sync calls for request-critical behavior**
- **events for decoupled reactions**

Example:

- Order placement creates the order synchronously
- Notification email happens via internal event

This gives clean separation without turning the monolith into fake distributed systems complexity.

Step 8: Standardize cross-cutting concerns centrally

A serious backend should standardize:

- auth/authz

- logging
- tracing/correlation ids
- exception handling
- validation
- transactions
- retries (for external calls)
- idempotency
- config management
- metrics/health checks

Best practice

Put cross-cutting mechanics in shared infrastructure, but keep business decisions in modules.

Example:

- shared: generic event bus abstraction
- module: OrderPlacedHandler

Not the other way around.

Step 9: Design transactions carefully

A huge advantage of modular monoliths is simpler transaction handling.

Because everything is in one app and usually one DB, you can often do:

- one transaction per request/use-case
- strong consistency
- simpler rollback semantics

Best practice

Keep transactions aligned with use cases and aggregate boundaries.

Example:

- placing an order + persisting order items + status creation = one transaction
- sending email = outside transaction / post-commit event

Common anti-pattern

Long transactions that include:

- external HTTP calls
- payment gateways

- message broker dependencies
- expensive queries

That increases lock contention and reliability issues.

Step 10: Build observability and testing around module boundaries

A modular monolith should not only be modular in code — it should be modular in **tests** and **observability** too.

Testing best practices

Use multiple layers:

- unit tests for domain rules
- module integration tests for use cases
- contract tests for module public APIs
- end-to-end tests for core business flows

Observability best practices

Tag logs/metrics by:

- module name
- use case
- request id / correlation id
- domain event name

That helps you diagnose complexity before even thinking about microservices.

Step 11: Make extraction possible, but don't prematurely optimize for it

A modular monolith should be **extractable**, not **pre-extracted**.

What that means:

- boundaries are explicit
- contracts exist
- module data ownership is respected
- side effects may already use events
- external integrations are behind adapters

If one day Billing needs independent scaling, then it can be extracted with much less pain.

Best practice

Design so extraction is possible.

Anti-pattern

Force distributed-system complexity on day 1 “just in case”.

A practical folder example

For a Node/NestJS-style project, something like this works well:

```
1  src/  
2    app/  
3      main.ts  
4      app.module.ts  
5  
6    shared/  
7      auth/  
8      config/  
9      logging/  
10     database/  
11     events/  
12     exceptions/  
13  
14    modules/  
15      identity/  
16        api/  
17        application/  
18        domain/  
19        infrastructure/  
20        contracts/  
21  
22      users/  
23        api/  
24        application/  
25        domain/  
26        infrastructure/  
27        contracts/  
28  
29      orders/  
30        api/
```

31	application/
32	domain/
33	infrastructure/
34	contracts/
35	
36	billing/
37	api/
38	application/
39	domain/
40	infrastructure/
41	contracts/

The key is not the exact folder names.

The key is **ownership + boundaries + dependency discipline**.

4) Pros

1. Operational simplicity

Much simpler than microservices:

- one deployment unit
 - fewer moving parts
 - easier local setup
 - easier CI/CD
-

2. Easier debugging

A request stays inside one process most of the time:

- easier stack traces
- easier breakpoints
- easier root cause analysis

No distributed tracing dependency just to understand basic flows.

3. Strong consistency is easier

With one DB and one transaction boundary:

- simpler ACID transactions
- less need for sagas/outbox from day 1

- fewer eventual consistency headaches
-

4. Faster development for small-to-medium teams

You spend more time shipping features and less time managing infrastructure.

5. Lower infrastructure cost

No need for:

- many service deployments
 - service mesh
 - complex orchestration
 - many environment configs
 - extensive distributed monitoring from the start
-

6. Better codebase coherence when done right

A good modular monolith gives you:

- explicit business boundaries
 - clean ownership
 - easier refactoring
 - easier onboarding
-

7. Easier path to microservices later

If your boundaries are well-designed, some modules can later be extracted.

5) Cons

1. One deployment unit can become a bottleneck

Even if only Billing changed, you deploy the whole application.

2. Shared runtime means imperfect isolation

A memory leak or CPU hotspot in one module can affect the whole app.

3. Team autonomy is lower than true service independence

Multiple teams touching one codebase can create friction if engineering discipline is weak.

4. Scaling is coarser

You scale the whole application, not one module independently.

If 90% of traffic is Catalog, but Billing is light, you still scale the app together unless you split later.

5. Boundary erosion is a constant risk

The biggest danger is not technical — it is human:

- shortcuts in imports
- direct table access
- bloated shared folder
- accidental circular dependencies

Without discipline, modular monolith becomes just “monolith with folders”.

6) When to use this architecture

A modular monolith is usually the right choice when:

1. You are in early or growth stage

You want to move fast without distributed-system overhead.

2. Your domain is substantial enough to need structure

You’ve outgrown a “simple CRUD app”, but you’re not yet at true microservices scale.

3. Strong consistency matters

Order creation, inventory updates, account actions, billing records — these are often easier in one transactional system.

4. Your team is small to medium

Roughly: one team or a few teams, not dozens of independently operating platform/product teams.

5. Your operational maturity is still developing

If you don't yet have strong capabilities in:

- service observability
- distributed tracing
- platform automation
- resilient async workflows
- production SRE practices

then microservices often hurt more than help.

6. You want a clean future migration path

Modular monolith is excellent when you want to delay distribution until there is a real reason.

7) Differences with microservices and when to use which one

Here's the core distinction:

Modular monolith solves code organization inside one system.

Microservices solve independent deployment and scaling across systems.

Comparison

Aspect	Modular Monolith	Microservices
Deployment	One deployable	Many independently deployable services
Runtime	Usually one app/process group	Multiple processes/services
Database	Often one DB, shared instance with ownership boundaries	Usually database per service
Communication	In-process calls, internal events	Network calls, messaging, async workflows
Transactions	Easier, often ACID in one DB	Harder, distributed consistency patterns needed
Operational complexity	Low to medium	High
Debugging	Easier	Harder due to network boundaries
Team autonomy	Medium	High when mature
Scaling	Coarse-grained	Fine-grained
Failure isolation	Lower	Higher in principle
Refactoring cost	Lower early	Higher upfront design cost

Infrastructure needs	Lower	Higher
----------------------	-------	--------

When modular monolith is better

Choose **modular monolith** when:

- product is still evolving fast
- domain boundaries are not fully stable yet
- team size is moderate
- strong consistency is important
- you want fast delivery with maintainability
- independent scaling is not yet critical
- you want architectural discipline without distributed systems pain

This is honestly the right answer for a lot of products.

When microservices are better

Choose **microservices** when there is a real, demonstrated need for:

1. Independent deployability

Different teams need to release independently without coordinating every deployment.

2. Independent scaling

One subsystem has drastically different traffic/resource profile.

Examples:

- search/catalog
- video processing
- analytics ingestion
- payment orchestration

3. Strong bounded contexts with stable ownership

If domains are mature and team ownership is clearly separated, microservices become more viable.

4. Different runtime/technology requirements

One service may need:

- Go for high throughput
- Python for ML

- Java for a core transaction engine
- Node.js for fast API composition

5. Organizational scale justifies it

The organization has enough maturity to handle:

- observability
 - platform engineering
 - API governance
 - service contracts
 - resiliency patterns
 - cost control
 - incident response at distributed-system scale
-

A senior-engineer rule of thumb

If the reason for microservices is:

- “It’s modern”
- “Big companies do it”
- “It will scale better someday”
- “We may need it later”

that is **not enough**.

If the reason is:

- “This subsystem needs independent deploys every day”
- “This module has radically different scaling needs”
- “Team autonomy is blocked by shared releases”
- “The domain boundaries are mature and stable”
- “We are ready operationally”

then microservices may be justified.

Final takeaway

The essence of modular monolith

A modular monolith is:

- **one deployable**

- many well-bounded business modules
- clear ownership
- controlled dependencies
- disciplined data access
- simple operations
- future extraction possible

The biggest misconception is:

“Monolith means messy.”

That is false.

A **badly designed monolith** is messy.

A **well-designed modular monolith** is often the most pragmatic and professional architecture for real backend systems.

My practical recommendation to you as a backend developer

When learning or building backend architecture, use this progression:

1. Learn **modular monolith** deeply first
2. Learn **DDD bounded contexts** and **clean/hexagonal module boundaries**
3. Learn **internal event-driven design** inside the monolith
4. Then learn **microservices** as a scaling/deployment evolution, not as the default starting point

That sequence gives you stronger engineering judgment.

If you want, I can next give you one of these:

1. a **NestJS modular monolith folder structure with sample modules**
2. a **real e-commerce example (Auth + Order + Billing + Notification)**
3. a **step-by-step migration path from modular monolith to microservices**
4. a **checklist to review whether your current backend is truly modular or just a regular monolith**